PACE Controller - Cross-Platform User and Technical Manual

Sebastiano Romi - LENS, University of Florence (UNIFI)

Version 1.0.3 - 2026

PACE Controller

User and technical manual for version **1.0.3**.

1. Scope

PACE Controller is a graphical application for classic Druck PACE 5000 and PACE 6000 pressure controllers. It supports Windows and Linux and communicates through Ethernet TCP or RS-232 serial links.

The application sends SCPI commands directly. Ethernet and RS-232 use the same device-control engine, automation state machine, validation rules, and interlocks. No NI-VISA, Druck USB driver, LabVIEW, or Internet access is required at runtime.

The original PowerShell/WinForms v0.3.1 remains unchanged as a legacy Windows implementation. The application described in this manual is a separate PySide6 codebase.

2. Safety notice

This software can command real pressurization, venting, and transitions between CONTROL and MEASURE. Before use:

- verify the pressure rating of the sample, cell, tubing, fittings, valves, and seals;
- verify SUPPLY, OUTLET, VENT, and reference connections;
- configure appropriate pressure and slew limits in the PACE itself;
- keep the PACE front panel and physical source shutoff accessible;
- test each workflow without a valuable sample and at low pressure;
- never rely on this application as the only pressure-protection layer.

The protections described below depend on valid telemetry, a functioning computer, and an intact communication link. They are not safety-rated or deterministic hardware interlocks.

3. Supported systems

3.1 Instruments

- classic Druck PACE 5000 without the E suffix;
- classic Druck PACE 6000 without the E suffix;
- control module 1 or 2, selected from the connection panel.

The SCPI implementation follows the Druck PACE SCPI Remote Communications Manual K0472.

3.2 Computers

- Windows 10 or Windows 11, x86-64;
- modern x86-64 Linux distributions;
- Python 3.10 or newer when running from source.

Precompiled release packages do not require Python on the target computer. The Linux package relies on the standard Qt/X11 runtime libraries normally present on a modern desktop distribution.

4. Connection methods

4.1 Ethernet

Configure the PACE Ethernet parameters as follows:

Parameter	Value
Address mode	Static
IP address	`192.168.10.2`
Subnet mask	`255.255.255.0`
Gateway	Empty or `0.0.0.0`
DNS	Empty or `0.0.0.0`
Access control	Open
SCPI port	TCP `5025`

The Ethernet transport sends every command with a CRLF line ending. Its final LF byte (ASCII 10) is the SCPI message terminator required by K0472, while the complete sequence preserves the validated legacy Windows implementation.

Connect the computer and PACE directly with an Ethernet cable. Modern interfaces normally support automatic crossover.

The application first attempts the configured address without changing the computer. If the PACE is unreachable and automatic configuration is enabled, it looks for exactly one safe dedicated wired adapter satisfying all of these conditions:

- physical wired interface;
- link state up;
- no default gateway;
- no unrelated non-link-local IPv4 address.

Only that adapter receives the temporary address `192.168.10.1/24`. If zero or multiple safe candidates exist, no adapter is changed. The address is removed when the application closes.

On Windows, changing an adapter requires administrator privileges. On Linux, the program uses `ip` and requests authorization through `pkexec` when needed. If automatic configuration is unavailable, configure the adapter manually.

4.2 RS-232

Select the PACE RS-232 communication interface, SCPI protocol, and CR message terminator. The recommended initial settings are:

Parameter	Initial value
Baud rate	`9600` bit/s
Data bits	`8`
Parity	None
Stop bits	`1`
Flow control	None
Terminator	CR

The GUI supports PACE baud rates from 2400 to 115200, None/Even/Odd parity, no flow control, XON/XOFF, and RTS/CTS. Values selected on the PC must exactly match the instrument.

A native RS-232 port or a USB-to-RS-232 converter can be used. On Windows the port appears as `COM1`, `COM2`, and so on. On Linux it normally appears as `/dev/ttyS0` or `/dev/ttyUSB0`.

Linux users may need serial-port group membership:

```
sudo usermod -aG dialout "$USER"
```

Log out and back in after changing group membership.

4.3 Offline simulator

The simulator accepts the same subset of SCPI commands used by the application and produces deterministic pressure telemetry. It is intended for:

- interface training;
- routine editing;
- automated tests;
- screenshot generation;
- verification without pressurizing a system.

The simulator is not a physical model and must not be used to predict real controller performance.

4.4 Other interfaces

The classic PACE documentation describes IEEE-488/GPIB, but support depends on a compatible controller and system driver. GPIB is not enabled in version 1.0.0. Direct USB/VISA support is also excluded because it would reintroduce platform-specific proprietary components. USB-to-RS-232 remains supported as a normal serial connection.

Only one interface should control the PACE at a time.

5. Starting the application

5.1 Windows package

Download and run `PACE-Controller-vX.Y.Z-Windows-x86_64.exe`. The executable is unsigned, so Windows SmartScreen may request confirmation. Administrator authorization is required when the program must configure Ethernet automatically.

5.2 Linux package

Extract `PACE-Controller-vX.Y.Z-Linux-x86_64.tar.gz`, then run:

```
./PACE-Controller
```

If execution permission was not retained during transfer:

```
chmod +x PACE-Controller
```

5.3 Source version

Create a Python virtual environment, install `requirements.txt`, and start the module:

```
python -m pace_controller
```

Use `python -m pace_controller --simulate` for hardware-free operation.

6. Connection panel

Choose Ethernet, RS-232, or Simulator from the transport selector.

For Ethernet enter the PACE IP and TCP port. For RS-232 select the detected serial port, baud rate, parity, and flow control. Then choose control module 1 or 2 and press **Connect**.

The software:

1. opens the selected transport;
2. requests `*IDN?`;
3. verifies a PACE identity;
4. clears previous SCPI errors;
5. sets pressure units to bar;
6. reads module range and pressure-control parameters;
7. begins telemetry polling.

Press **Disconnect** to stop automation, request MEASURE, close the communication channel, and restore temporary network configuration.

The bilingual **Help** menu contains **Report a problem**, which opens the public GitHub issue tracker, and **About PACE Controller**, which shows the version, author, institutional affiliation, contact email, project link, and independence notice.

7. Main display

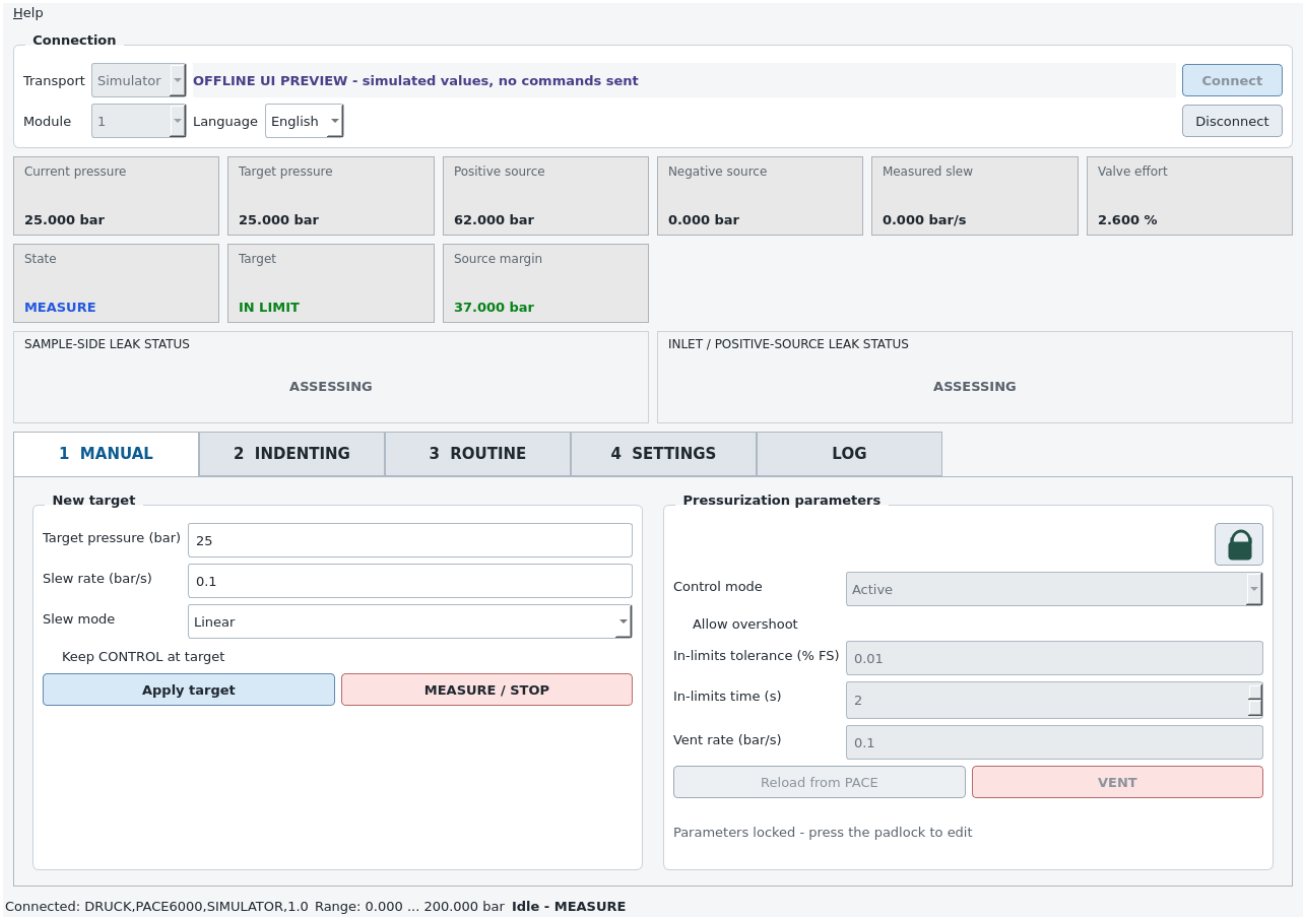

Figure 1: Real interface generated in offline simulator mode

The image above is produced by the actual application during the release pipeline. Screenshot mode selects the offline simulator, so it never sends commands to an instrument.

Nine telemetry cards are always arranged on two rows:

- current pressure;
- pressure target;

- positive and negative source pressure;
- measured slew rate;
- valve effort;
- CONTROL or MEASURE state;
- in-limits state;
- positive-source margin.

Pressure, target, source, slew, valve-effort, and margin values are displayed with three decimal places. CSV telemetry retains the full numeric values received and parsed by the application; visual formatting does not reduce stored precision.

Two prominent panels immediately below the telemetry display sample-side and positive-inlet leak status. The operating pages are MANUAL, INDENTING, ROUTINE, SETTINGS, and LOG.

8. Manual pressure control

Enter:

- target pressure in bar;
- slew rate in bar/s;
- Linear or Maximum slew mode;
- whether CONTROL should be maintained after reaching the target.

The keep-CONTROL option is disabled by default. If it remains disabled, the software requests MEASURE as soon as the PACE reports in-limits. Press **MEASURE / STOP** at any time to cancel active automation and request MEASURE.

9. Protected pressurization parameters

The pressurization panel is read-only by default. Press the padlock button to display a danger-area warning. Fields are unlocked only after explicit confirmation.

Protected parameters are:

- **Control mode:** Active, Passive, or Gauge;
- **Overshoot:** PACE overshoot option;
- **In-limits tolerance:** percentage of full scale used to declare the target reached;
- **In-limits time:** required stable duration;
- **Vent rate:** controlled vent rate in bar/s;
- **VENT:** command initiating controlled venting.

VENT has a separate confirmation even when the panel is unlocked. The panel automatically locks after disconnection.

10. Indenting cycle

The indenting page executes a fixed sequence:

1. set the selected slew;
2. enter CONTROL and reach the requested pressure;
3. wait for the PACE in-limits indication;
4. maintain CONTROL for 120 seconds;
5. command zero bar with the same slew;
6. wait for in-limits at zero;
7. request MEASURE.

Stopping the cycle requests MEASURE immediately.

11. Programmable routines

Each table row contains:

- target pressure in bar;
- slew rate in bar/s;
- dwell time after in-limits in seconds;
- an optional note.

Rows execute from top to bottom. Dwell begins only after the PACE reports in-limits. Add or remove rows with the buttons above the table. Save and load routines as JSON.

At completion the software requests MEASURE unless **Keep CONTROL after final step** was explicitly selected. Timeouts and user interruption request MEASURE.

12. Software protection rules

12.1 Large target change

An upward target change of at least 10 bar relative to the previous pressure or routine step requires explicit confirmation.

12.2 High slew rate

A slew greater than 0.5 bar/s or selection of Maximum mode requires explicit confirmation before a pressure change is submitted.

12.3 Module range

Targets below the detected minimum or above the detected maximum module pressure are blocked.

12.4 Positive-source margin

The source margin is

$$P_{\text{margin}} = P_{\text{source},+} - P_{\text{current}}.$$

CONTROL cannot start if the measured margin is below 2.0 bar or if a requested target would leave less than 2.0 bar.

If the margin falls below 2.0 bar during CONTROL, the software:

1. cancels manual automation, indenting, or the active routine;
2. sends the MEASURE command;
3. displays a critical warning;
4. records the event in the log.

After an intervention, CONTROL remains blocked until the margin reaches at least 2.2 bar. This hysteresis avoids repeated operation at the boundary.

12.5 Telemetry loss

If required telemetry is lost during CONTROL, the program attempts to send MEASURE immediately and stops automation. If communication prevents confirmation, use the PACE front panel and physical source controls immediately.

13. Leak monitoring

13.1 Signals

Two independent rolling histories are evaluated:

- **sample side:** current controlled pressure;
- **inlet side:** positive source pressure reported by COMP1.

Only decreases are considered pressure losses. Increases are clipped to zero loss.

13.2 Operating condition

Histories are collected only in MEASURE with no active automation. During CONTROL they are cleared and both panels show **ASSESSMENT PAUSED (CONTROL)**. After returning to MEASURE, assessment restarts.

13.3 Calculation and default classes

A linear least-squares slope is calculated over the rolling history. With default settings:

Display	Colour	Equivalent fitted loss rate
NO LEAK	Green	`<=0.0005 bar/min` after 10 min
WARNING: slight leak	Yellow	`>0.0005` and `<=0.001 bar/min` after 5 min
WARNING: pressure leak	Orange	`>0.001` and `<=0.005 bar/min` after 1 min
WARNING: SIGNIFICANT PRESSURE LEAK	Red	`>0.005 bar/min`

The SETTINGS page permits editing the reference drop and green, yellow, and orange times. Required ordering is `green > yellow > orange > 0`.

A pressure trend does not uniquely identify a physical leak. Thermal equilibration, regulator hysteresis, pressure-medium behaviour, and sensor noise can produce similar signals. Confirm warnings using an appropriate laboratory leak-test procedure.

14. Data and logs

The application writes:

- `PACE_controller_data.csv`: timestamped full-precision telemetry;
- `PACE_controller_log.txt`: commands, connection events, errors, and interlocks;
- `PACE_controller_settings.json`: language, connection, and leak preferences;
- user-selected JSON routine files.

Default folders are `%LOCALAPPDATA%\PACE Controller` on Windows and `${XDG_DATA_HOME:-~/.local/share}/pace-control` on Linux. The SETTINGS page provides a button to open the active folder.

15. Troubleshooting

PACE cannot be reached over Ethernet

Verify the static IP, subnet, cable, Ethernet LEDs, TCP port 5025, Access Control = Open, and firmware Ethernet support. Confirm that no unrelated adapter already uses `192.168.10.0/24`.

No safe Ethernet adapter is found

Disconnect unused wired interfaces or configure the known PACE adapter manually. The program intentionally refuses ambiguous selection.

No serial ports are listed

Verify cable and USB-to-RS-232 driver, then press **Refresh**. On Linux inspect `/dev/ttyUSB*` and confirm membership in the `dialout` group.

Serial connection times out

Ensure baud rate, parity, flow control, SCPI protocol, and CR terminator match the PACE. Confirm that the selected port is not open in another program.

Communication is lost during CONTROL

Use the PACE front panel immediately, select MEASURE if necessary, verify the physical system, and then inspect the cable and log. Software cannot guarantee command delivery after communication loss.

Leak status remains in assessment

Confirm the PACE is in MEASURE and wait for the configured history interval. CONTROL, venting, module changes, threshold changes, and reconnection restart the history.

16. Technical architecture

The application is divided into:

- a Qt/PySide6 graphical interface;
- a background service serializing every device operation;
- TCP, serial, and simulator transport classes implementing the same query/write interface;
- a shared SCPI controller and automation state machine;
- independent leak-monitoring and persistent-storage modules.

All device I/O occurs in one background thread so TCP and serial timeouts do not freeze the GUI. UI requests are queued, and telemetry and alarms return through thread-safe Qt signals.

The simulator and transport-independent tests allow most behaviour to be checked without hardware. Real low-pressure tests remain required before operational use.

17. Release and reproducibility

GitHub Actions perform:

- Python syntax and unit tests on Windows and Linux;
- real offscreen Qt screenshot generation;
- standalone Windows and Linux builds with PyInstaller;
- manual PDF generation;
- source packaging and SHA-256 checksums;
- semantic tagging and release publication.

The complete source and workflows are included in the repository. Binaries remain unsigned and must be handled according to local IT policy.

18. References and project status

1. Druck, *PACE SCPI Remote Communications Manual*, K0472: https://druck.com/wp-content/uploads/2026/05/K0472-PACE-SCPI-Remote-Communications-Manual-EN.pdf
2. Druck classic PACE 5000 support page: https://druck.com/product/pace-5000/

PACE Controller is independent software and is not an official Druck product. It is released under the MIT License.

Author and contact

- Sebastiano Romi
- European Laboratory for Non-Linear Spectroscopy (LENS)
- University of Florence (UNIFI)
- romi@lens.unifi.it

Development used AI-assisted programming. Validate the application on a safe, unloaded, low-pressure setup and report unexpected behaviour with the logfile, PACE model, module, firmware version, connection type, and exact reproduction steps.